```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import requests
from io import BytesIO
from IPython.display import display, Markdown

sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", None)

print("All libraries imported successfully.")
```

```
All libraries imported successfully.
```

```
import pandas as pd
import requests
from io import BytesIO

dataset_urls = [
    "https://raw.githubusercontent.com/Wunmi-O/Superstore/master/SampleSuper
    "https://raw.githubusercontent.com/leonism/sample-superstore/master/data
    "https://raw.githubusercontent.com/Victory-Data/Superstore-Sales-Dataset
]

df = None

for url in dataset_urls:
    try:
        response = requests.get(url, timeout=30)
        response.raise_for_status()

        temp_df = pd.read_csv(BytesIO(response.content), encoding="latin1")

        if len(temp_df) > 0 and "Sales" in temp_df.columns:
            df = temp_df
            print("Dataset downloaded successfully!")
            print("Source:", url)
            break

    except Exception as error:
        print("Source unavailable:", url)
        print("Reason:", error)

if df is None:
    raise RuntimeError(
        "All online sources failed. Please check your internet connection "
        "or download a CSV from one of the source links."
    )

print("\nDataset shape:", df.shape)
```

```
print("\nColumns:", df.columns.tolist())
display(df.head())
```

```
Dataset downloaded successfully!
Source: https://raw.githubusercontent.com/Wunmi-O/Superstore/master/SampleSup

Dataset shape: (9994, 13)

Columns: ['Ship Mode', 'Segment', 'Country', 'City', 'State', 'Postal Code',
```

	Ship Mode	Segment	Country	City	State	Postal Code	Region	Category
0	Second Class	Consumer	United States	Henderson	Kentucky	42420	South	Furniture
1	Second Class	Consumer	United States	Henderson	Kentucky	42420	South	Furniture
2	Second Class	Corporate	United States	Los Angeles	California	90036	West	Office Supplies
3	Standard Class	Consumer	United States	Fort Lauderdale	Florida	33311	South	Furniture
4	Standard Class	Consumer	United States	Fort Lauderdale	Florida	33311	South	Office Supplies

```
if df is not None:
    display(df.head())
```

	Ship Mode	Segment	Country	City	State	Postal Code	Region	Category
0	Second Class	Consumer	United States	Henderson	Kentucky	42420	South	Furniture
1	Second Class	Consumer	United States	Henderson	Kentucky	42420	South	Furniture
2	Second Class	Corporate	United States	Los Angeles	California	90036	West	Office Supplies
3	Standard Class	Consumer	United States	Fort Lauderdale	Florida	33311	South	Furniture
4	Standard Class	Consumer	United States	Fort Lauderdale	Florida	33311	South	Office Supplies

```
if df is not None:
    print("Dataset shape:", df.shape)
    print("\nColumn names:")
    print(df.columns.tolist())

    print("\nDataset information:")
    df.info()
```

```
Dataset shape: (9994, 13)

Column names:
['Ship Mode', 'Segment', 'Country', 'City', 'State', 'Postal Code', 'Region',

Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 13 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Ship Mode     9994 non-null   object
 1   Segment       9994 non-null   object
 2   Country       9994 non-null   object
 3   City          9994 non-null   object
 4   State         9994 non-null   object
 5   Postal Code   9994 non-null   int64
 6   Region        9994 non-null   object
 7   Category      9994 non-null   object
 8   Sub_Category  9994 non-null   object
 9   Sales         9994 non-null   float64
 10  Quantity      9994 non-null   float64
 11  Discount      9994 non-null   float64
 12  Profit        9994 non-null   float64
dtypes: float64(4), int64(1), object(8)
memory usage: 1015.1+ KB
```

```
if df is not None:
    df.columns = df.columns.str.strip()

    print("Missing values before cleaning:")
    display(df.isnull().sum().to_frame("Missing Values"))

    print("\nDuplicate rows:", df.duplicated().sum())
```

```
Missing values before cleaning:
```

	Missing Values
Ship Mode	0
Segment	0
Country	0
City	0
State	0
Postal Code	0
Region	0
Category	0
Sub_Category	0
Sales	0
Quantity	0
Discount	0
Profit	0

```
Duplicate rows: 17
```

```
if df is not None:
    df = df.drop_duplicates().copy()

    # Remove rows that have no usable values in key business fields.
    key_columns = [
        col for col in ["Sales", "Profit", "Category", "Region"]
        if col in df.columns
    ]

    if key_columns:
        df = df.dropna(subset=key_columns)

    print("Data cleaning completed.")
    print("Remaining rows:", len(df))
    print("Remaining columns:", len(df.columns))
    print("Missing values remaining:", int(df.isnull().sum().sum()))
```

```
Data cleaning completed.
Remaining rows: 9977
Remaining columns: 13
Missing values remaining: 0
```

```
if df is not None:
    if "Order Date" in df.columns:
        df["Order Date"] = pd.to_datetime(
            df["Order Date"], errors="coerce"
```

```
        )

    for col in ["Sales", "Profit", "Quantity", "Discount"]:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors="coerce")

    print("Data types prepared.")
    display(df.dtypes.to_frame("Data Type"))
```

```
Data types prepared.
```

	Data Type
Ship Mode	object
Segment	object
Country	object
City	object
State	object
Postal Code	int64
Region	object
Category	object
Sub_Category	object
Sales	float64
Quantity	float64
Discount	float64
Profit	float64

```
if df is not None:
    display(df.describe(include="all").T)
```

	count	unique	top	freq	mean	std	
Ship Mode	9977	4	Standard Class	5955	NaN	NaN	
Segment	9977	3	Consumer	5183	NaN	NaN	
Country	9977	1	United States	9977	NaN	NaN	
City	9977	531	New York City	914	NaN	NaN	
State	9977	49	California	1996	NaN	NaN	
Postal Code	9977.0	NaN	NaN	NaN	55154.964117	32058.266816	10
Region	9977	4	West	3193	NaN	NaN	
Category	9977	3	Office Supplies	6012	NaN	NaN	
Sub_Category	9977	17	Binders	1522	NaN	NaN	
Sales	9977.0	NaN	NaN	NaN	230.148924	623.721439	
Quantity	9977.0	NaN	NaN	NaN	3.790719	2.226657	
Discount	9977.0	NaN	NaN	NaN	0.156278	0.206455	
Profit	9977.0	NaN	NaN	NaN	28.690207	234.457935	-659

```
if df is not None:
    if "Sales" not in df.columns:
        raise ValueError("The dataset must contain a Sales column.")

    total_sales = df["Sales"].sum()
    total_orders = (
        df["Order ID"].nunique()
        if "Order ID" in df.columns
        else len(df)
    )

    total_profit = (
        df["Profit"].sum()
        if "Profit" in df.columns
        else np.nan
    )

    total_quantity = (
        df["Quantity"].sum()
        if "Quantity" in df.columns
        else np.nan
    )

    profit_margin = (
        (total_profit / total_sales) * 100
```

```
            if pd.notna(total_profit) and total_sales != 0
            else np.nan
        )

        print("KEY PERFORMANCE INDICATORS")
        print("-" * 40)
        print(f"Total Sales: {total_sales:,.2f}")
        print(f"Order Count: {total_orders:,}")
        print(f"Total Profit: {total_profit:,.2f}")
        print(f"Total Quantity Sold: {total_quantity:,.0f}")
        print(f"Profit Margin: {profit_margin:.2f}%")
```

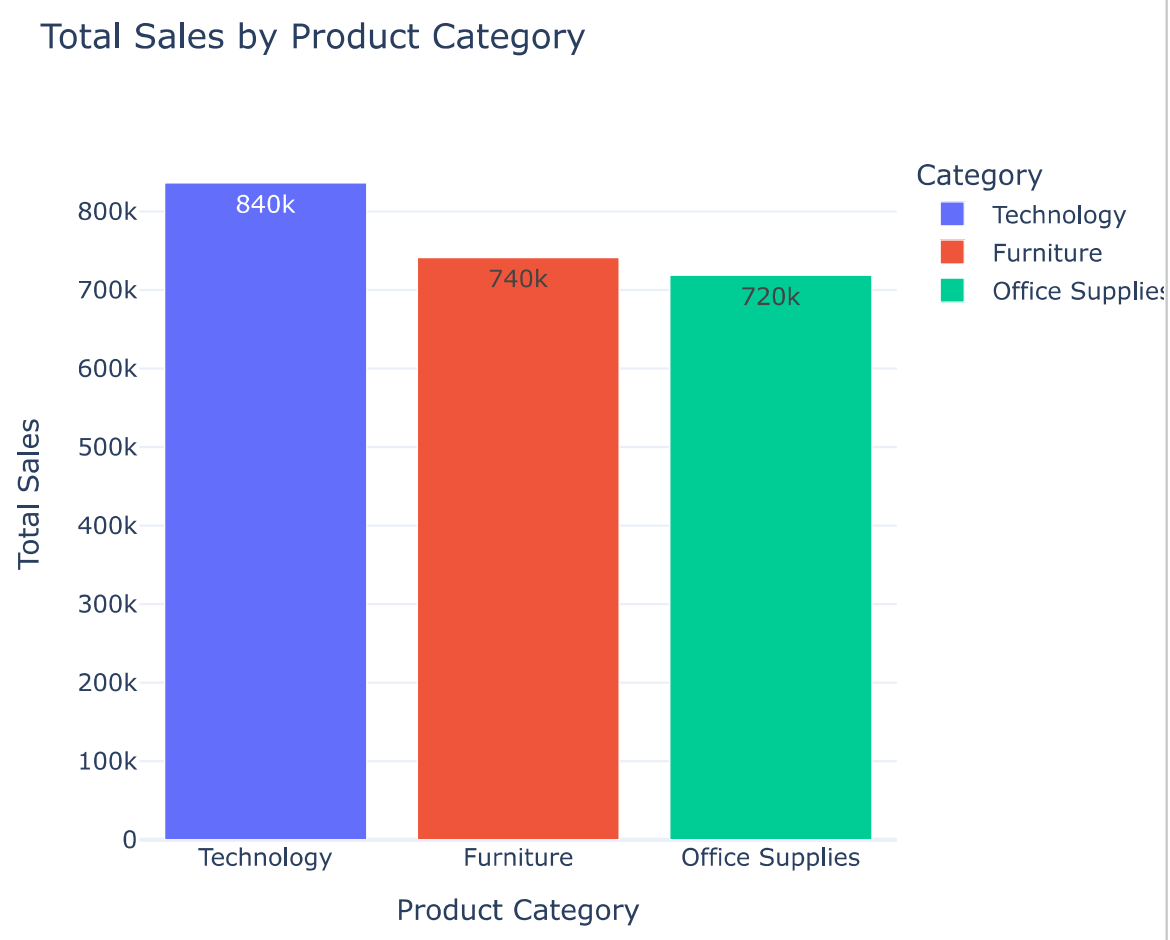
```
KEY PERFORMANCE INDICATORS
----------------------------------------
Total Sales: 2,296,195.81
Order Count: 9,977
Total Profit: 286,242.20
Total Quantity Sold: 37,820
Profit Margin: 12.47%
```

```
if df is not None and "Category" in df.columns:
    category_sales = (
        df.groupby("Category", as_index=False)["Sales"]
        .sum()
        .sort_values("Sales", ascending=False)
    )

    fig = px.bar(
        category_sales,
        x="Category",
        y="Sales",
        color="Category",
        title="Total Sales by Product Category",
        text_auto=".2s",
        template="plotly_white"
    )

    fig.update_layout(
        xaxis_title="Product Category",
        yaxis_title="Total Sales"
    )
    fig.show()

    display(category_sales)
```



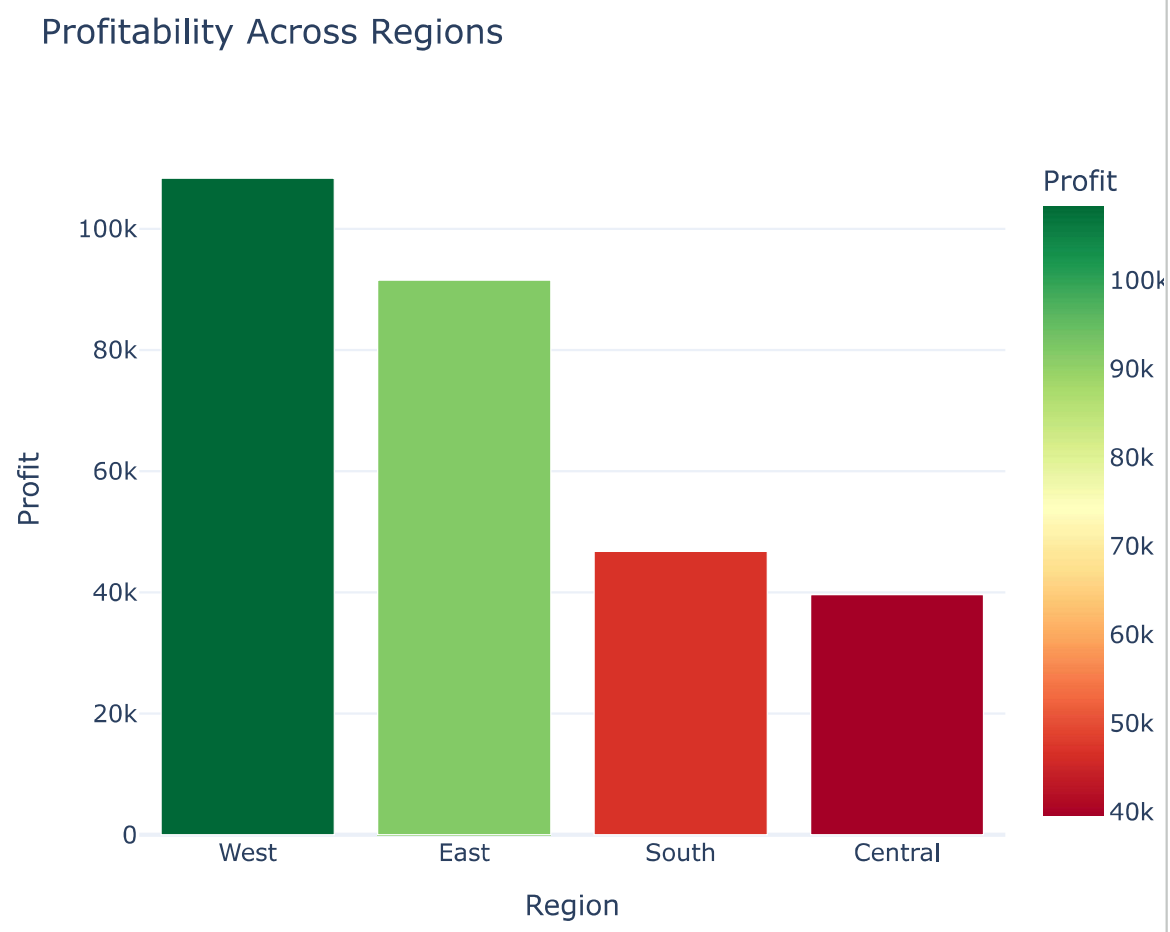
	Category	Sales
2	Technology	836154.10
0	Furniture	741306.50
1	Office Supplies	718735.21

Next steps: Generate code with category_sales | New interactive sheet

```
if df is not None and "Region" in df.columns and "Profit" in df.columns:
    region_profit = (
        df.groupby("Region", as_index=False)["Profit"]
        .sum()
        .sort_values("Profit", ascending=False)
    )

    fig = px.bar(
        region_profit,
        x="Region",
        y="Profit",
        color="Profit",
        title="Profitability Across Regions",
        template="plotly_white",
        color_continuous_scale="RdYlGn"
    )
```

```
fig.show()
display(region_profit)
```



	Region	Profit
3	West	108330.15
1	East	91506.37
2	South	46749.71
0	Central	39655.97

Next steps: Generate code with region_profit | New interactive sheet

```
if df is not None and "Order Date" in df.columns:
    monthly_sales = (
        df.dropna(subset=["Order Date"])
        .assign(Month=lambda x: x["Order Date"].dt.to_period("M").dt.to_time
        .groupby("Month", as_index=False)["Sales"]
        .sum()
        .sort_values("Month")
    )

    fig = px.line(
        monthly_sales,
```

```
            x="Month",
            y="Sales",
            markers=True,
            title="Monthly Sales Trend",
            template="plotly_white"
        )

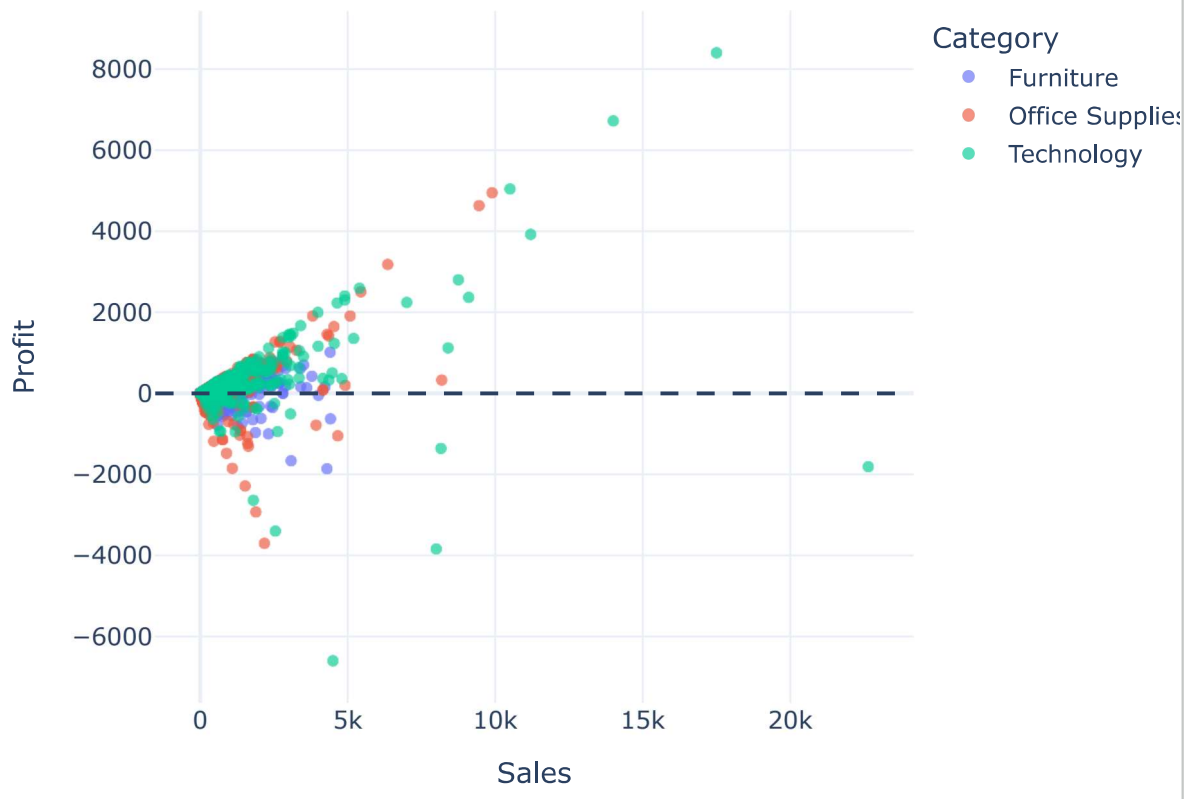
        fig.update_layout(
            xaxis_title="Month",
            yaxis_title="Total Sales"
        )
        fig.show()

        display(monthly_sales.head())
    else:
        print("Monthly trend unavailable: Order Date column not found.")
```

```
Monthly trend unavailable: Order Date column not found.
```

```
if df is not None and {"Sales", "Profit"}.issubset(df.columns):
    fig = px.scatter(
        df,
        x="Sales",
        y="Profit",
        color="Category" if "Category" in df.columns else None,
        hover_data=[
            col for col in ["Region", "Sub-Category", "Discount"]
            if col in df.columns
        ],
        title="Relationship Between Sales and Profit",
        template="plotly_white",
        opacity=0.65
    )

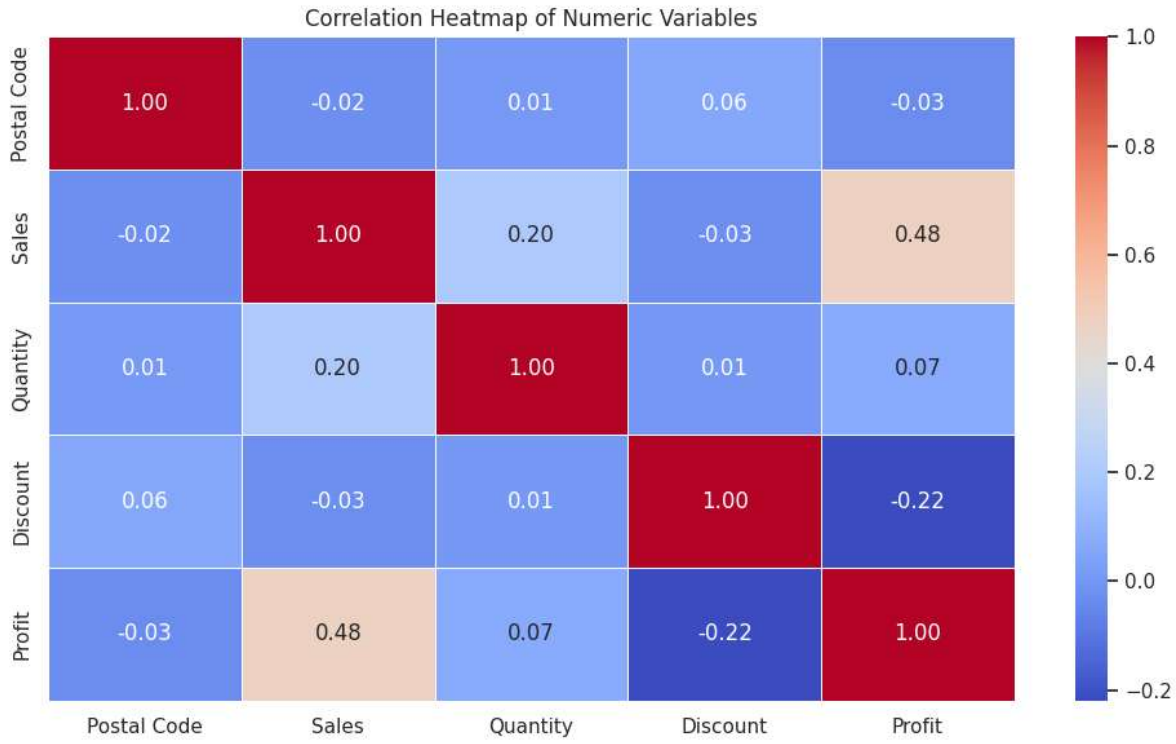
    fig.add_hline(y=0, line_dash="dash")
    fig.show()
```




```
if df is not None:
    numeric_df = df.select_dtypes(include=np.number)

    if numeric_df.shape[1] >= 2:
        correlation = numeric_df.corr()

        plt.figure(figsize=(10, 6))
        sns.heatmap(
            correlation,
            annot=True,
            cmap="coolwarm",
            fmt=".2f",
            linewidths=0.5
        )
        plt.title("Correlation Heatmap of Numeric Variables")
        plt.tight_layout()
        plt.show()
    else:
        print("Not enough numeric columns for a correlation heatmap.")
```




```
if df is not None and "Category" in df.columns:
    category_metrics = df.groupby("Category").agg(
        Total_Sales=("Sales", "sum"),
        Total_Transactions=("Sales", "count")
    )

    if "Profit" in df.columns:
        category_metrics["Total_Profit"] = (
            df.groupby("Category")["Profit"].sum()
        )

    display(category_metrics.sort_values("Total_Sales", ascending=False))

    best_category = category_metrics["Total_Sales"].idxmax()
    print(f"Highest-sales category: {best_category}")

    if "Total_Profit" in category_metrics.columns:
        worst_profit_category = category_metrics["Total_Profit"].idxmin()
        print(f"Lowest-profit category: {worst_profit_category}")
```

Category	Total_Sales	Total_Transactions	Total_Profit
Technology	836154.10	1847	145455.66
Furniture	741306.50	2118	18421.79
Office Supplies	718735.21	6012	122364.75

```
Highest-sales category: Technology
Lowest-profit category: Furniture
```

```
if df is not None and {"Sub-Category", "Profit"}.issubset(df.columns):
    subcategory_profit = (
        df.groupby("Sub-Category", as_index=False)["Profit"]
        .sum()
        .sort_values("Profit")
    )

    loss_making = subcategory_profit[
        subcategory_profit["Profit"] < 0
    ]

    print("Loss-making subcategories:")
    display(loss_making)

    fig = px.bar(
        subcategory_profit,
        x="Sub-Category",
        y="Profit",
        color="Profit",
        title="Profit by Subcategory",
        template="plotly_white",
        color_continuous_scale="RdYlGn"
    )

    fig.add_hline(y=0, line_dash="dash")
    fig.update_layout(xaxis_tickangle=-45)
    fig.show()
```

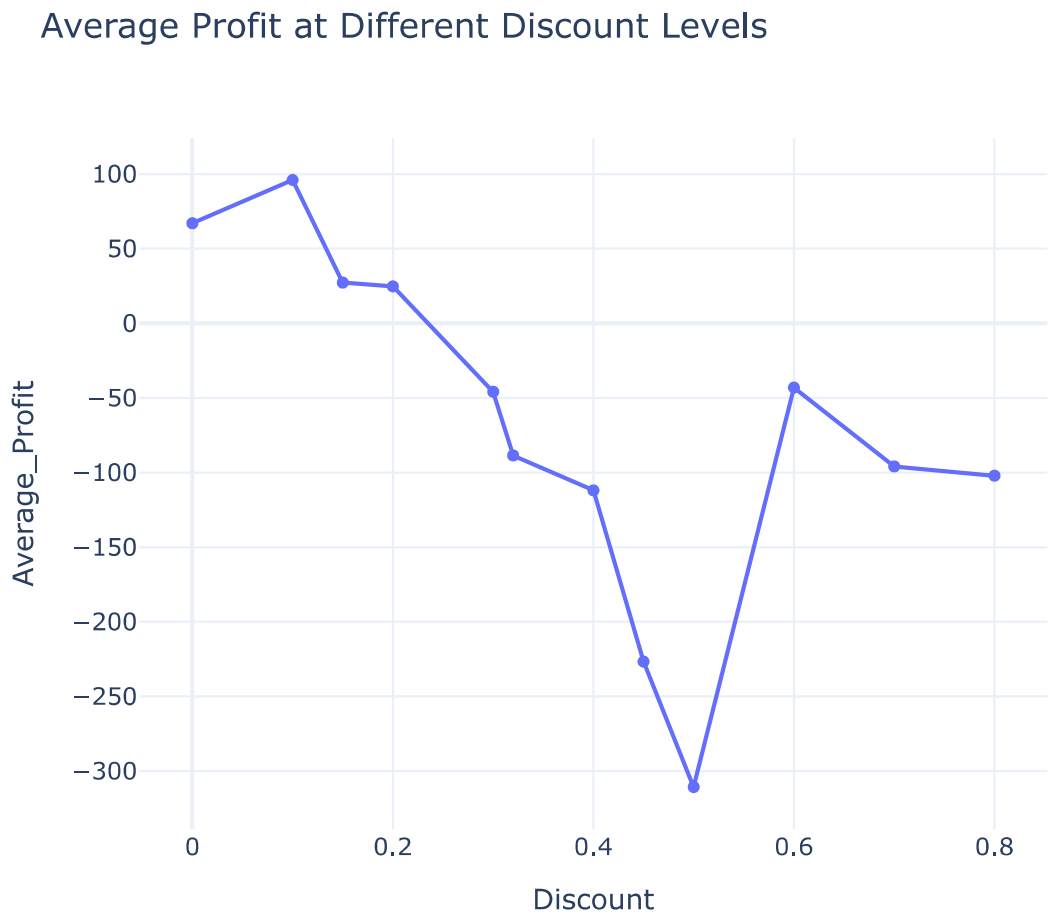
```
if df is not None and {"Discount", "Profit"}.issubset(df.columns):
    discount_analysis = (
        df.groupby("Discount", as_index=False)
        .agg(
            Average_Profit=("Profit", "mean"),
            Total_Profit=("Profit", "sum"),
            Number_of_Records=("Profit", "size")
        )
        .sort_values("Discount")
    )

    display(discount_analysis)

    fig = px.line(
        discount_analysis,
        x="Discount",
        y="Average_Profit",
        markers=True,
        title="Average Profit at Different Discount Levels",
        template="plotly_white"
    )

    fig.show()
```

	Discount	Average_Profit	Total_Profit	Number_of_Records
0	0.00	67.024165	320844.68	4787
1	0.10	96.055426	9029.21	94
2	0.15	27.288077	1418.98	52
3	0.20	24.721454	90307.47	3653
4	0.30	-45.828673	-10357.28	226
5	0.32	-88.561481	-2391.16	27
6	0.40	-111.927573	-23057.08	206
7	0.45	-226.647273	-2493.12	11
8	0.50	-310.704697	-20506.51	66
9	0.60	-43.077101	-5944.64	138
10	0.70	-95.874306	-40075.46	418
11	0.80	-102.116689	-30532.89	299



Next steps: Generate code with `discount_analysis` New interactive sheet

```
if df is not None:
```

```
print("DATA STORY: BUSINESS PERFORMANCE")
print("=" * 50)

print(f"Total sales were {total_sales:,.2f}.")

if pd.notna(total_profit):
    print(f"Total profit was {total_profit:,.2f}.")
    print(f"Overall profit margin was {profit_margin:.2f}%.")

if "Category" in df.columns:
    print(
        f"The highest-sales category was "
        f"{df.groupby('Category')['Sales'].sum().idxmax()}."
    )

if "Region" in df.columns and "Profit" in df.columns:
    print(
        f"The highest-profit region was "
        f"{df.groupby('Region')['Profit'].sum().idxmax()}."
    )

if "Sub-Category" in df.columns and "Profit" in df.columns:
    print(
        f"The lowest-profit subcategory was "
        f"{df.groupby('Sub-Category')['Profit'].sum().idxmin()}."
    )

print("\nBUSINESS RECOMMENDATIONS")
print("1. Investigate loss-making product categories and subcategories.")
print("2. Compare regional profitability before allocating more budget.")
print("3. Review discount policies using profit and sales data.")
print("4. Track monthly sales to identify seasonality.")
print("5. Monitor KPIs regularly using an interactive dashboard.")
```

```
DATA STORY: BUSINESS PERFORMANCE
==================================================
Total sales were 2,296,195.81.
Total profit was 286,242.20.
Overall profit margin was 12.47%.
The highest-sales category was Technology.
The highest-profit region was West.

BUSINESS RECOMMENDATIONS
1. Investigate loss-making product categories and subcategories.
2. Compare regional profitability before allocating more budget.
3. Review discount policies using profit and sales data.
4. Track monthly sales to identify seasonality.
5. Monitor KPIs regularly using an interactive dashboard.
```

```
if df is not None:
    from IPython.display import display, HTML

    display(HTML("""
    <h2 style="text-align:center;">
        Global Superstore – Business Intelligence Dashboard
    </h2>
```

```
<p style="text-align:center;">
    Interactive analysis of sales, profitability, and business performa
</p>
"""))

if "Category" in df.columns:
    fig1 = px.bar(
        df.groupby("Category", as_index=False)["Sales"].sum(),
        x="Category",
        y="Sales",
        title="Sales by Category"
    )
    fig1.show()

if "Region" in df.columns and "Profit" in df.columns:
    fig2 = px.bar(
        df.groupby("Region", as_index=False)["Profit"].sum(),
        x="Region",
        y="Profit",
        title="Profit by Region"
    )
    fig2.show()

if "Order Date" in df.columns:
    monthly = (
        df.dropna(subset=["Order Date"])
        .assign(Month=lambda x: x["Order Date"].dt.to_period("M").dt.to
        .groupby("Month", as_index=False)["Sales"]
        .sum()
    )

    fig3 = px.line(
        monthly,
        x="Month",
        y="Sales",
        markers=True,
        title="Monthly Sales"
    )
    fig3.show()
```

Global Superstore — Business Intelligence Dashboard

Interactive analysis of sales, profitability, and business performance